

数据分析资料

一、NumPy + Pandas + Matplotlib/Seaborn (6 学时)

1.环境配置 (10 分钟, 正式内容前完成)

讲解内容

同学们, 在开始数据分析之前, 我们需要先配置好 Python 环境。本次实训使用 Windows 操作系统, Python 版本为 3.11.10, 并使用 `uv` 这个快速的包管理工具来管理依赖。

为什么要用虚拟环境?

虚拟环境可以为每个项目创建独立的 Python 包空间, 避免不同项目之间的库版本冲突。例如项目 A 需要 pandas 1.5, 项目 B 需要 pandas 2.0, 虚拟环境可以分别隔离。

您说得对, `uv` 提供了更简洁的工作流: 使用 `uv init` 初始化项目, `uv add` 添加依赖, 无需手动激活虚拟环境。以下是修正后的环境配置步骤 (Windows + Python 3.11.10):

环境配置 (使用 uv, 推荐方式)

讲解内容 (约 5 分钟)

`uv` 是一个极快的 Python 包管理器, 内置虚拟环境管理。推荐使用 `uv init` + `uv add` 的方式, 全程无需手动激活虚拟环境。

步骤:

1. 安装 uv (如未安装)

在 PowerShell 中执行:

```
Bash
```

```
1 powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

安装完成后关闭并重新打开终端。

2. 创建项目目录并初始化

Bash

```
1 mkdir superstore_analysis
2 cd superstore_analysis
3 uv init
```

此时会自动生成 `pyproject.toml` 和虚拟环境（位于 `.venv` 目录）。

3. 指定 Python 版本并添加依赖

Bash

```
1 uv python pin 3.11.10          # 锁定 Python 3.11.10
2 uv add numpy pandas matplotlib seaborn scikit-learn
```

`uv add` 会自动将依赖写入 `pyproject.toml` 并安装到虚拟环境中。

4. 运行 Python 代码

无需手动激活虚拟环境，直接使用 `uv run`：

Bash

```
1 uv run python script.py
```

或在 VS Code 中安装 Python 扩展后，选择 `.venv` 作为解释器即可。

验证：

Bash

```
1 uv run python -c "import numpy, pandas, matplotlib, seaborn; print('环境就绪')"
```

对比传统方式：

- 传统： `uv venv --python 3.11.10` → 激活 → `uv pip install ...`

- 推荐: `uv init` → `uv add ...` → `uv run python` (无需激活)

后续所有案例中, 请使用 `uv run python 文件名.py` 执行代码。

2. NumPy 基础

讲解内容 (约 20 分钟)

1. NumPy 是什么?

NumPy 是 Python 科学计算的基础库。它提供了一个高性能的多维数组对象 `ndarray`, 以及用于快速操作数组的函数。相比 Python 原生的列表 (list), NumPy 数组在存储和计算上更高效, 尤其是对于大规模数值数据。例如, 对一个包含百万个元素的数组进行加法, NumPy 可以比 Python 循环快几十倍甚至上百倍。

2. 创建 NumPy 数组

- `np.array([1, 2, 3])`: 从列表或元组创建数组。
- `np.arange(start, stop, step)`: 类似 Python 的 `range`, 但返回数组。例如 `np.arange(0, 10, 2)` → `[0, 2, 4, 6, 8]`。
- `np.linspace(start, stop, num)`: 在区间内生成等间隔的 `num` 个点。例如 `np.linspace(0, 1, 5)` → `[0, 0.25, 0.5, 0.75, 1]`。
- `np.zeros(shape)` 和 `np.ones(shape)`: 创建全 0 或全 1 数组, `shape` 是一个元组, 如 `(3, 4)` 表示 3 行 4 列。
- `np.random` 模块: 生成随机数组, 例如 `np.random.randint(1, 100, size=10)` 生成长度为 10 的随机整数数组。

3. 数组属性

- `shape`: 数组的维度形状, 如 `(3, 4)` 表示 3 行 4 列。
- `dtype`: 数组中元素的数据类型, 如 `int64`, `float64`。
- `ndim`: 数组的维数 (轴的数量)。
- `size`: 数组中元素的总个数。

4. 改变数组形状

- `reshape(new_shape)`: 返回一个新的视图 (不复制数据), 但要求元素个数不变。例如 `np.arange(12).reshape(3, 4)` 将一维数组变成 3 行 4 列。

5. 为什么用数组而不是列表?

- **向量化运算**: 可以对整个数组同时做运算, 不用写循环。
- **内存连续**: 存储效率高。
- **丰富的函数库**: 线性代数、随机数、傅里叶变换等。

代码案例（约 25 分钟）

案例名称：超市订单的销售数据数组化

目的：练习创建 NumPy 数组、查看属性、改变形状。

数据：模拟 10 个订单的销售额、数量、折扣率。

Python

```
1 import numpy as np
2
3 # 1. 创建一维数组：10个订单的销售额（单位：元）
4 sales = np.array([120.5, 45.0, 300.0, 27.5, 89.9, 210.0, 55.3, 450.0, 7
5 8.0, 150.0])
6 print("销售额数组:", sales)
7
8 # 2. 使用 arange 生成订单编号（从1001开始，步长1）
9 order_ids = np.arange(1001, 1011) # 1001到1010
10 print("订单编号:", order_ids)
11
12 # 3. 使用 linspace 生成10个订单的折扣率（从0到0.3均匀分布）
13 discounts = np.linspace(0, 0.3, 10)
14 print("折扣率:", discounts)
15
16 # 4. 生成随机数量（1到9之间）的数组
17 quantities = np.random.randint(1, 10, size=10)
18 print("购买数量:", quantities)
19
20 # 5. 将销售额和数量组合成二维数组（10行2列）
21 sales_qty = np.column_stack((sales, quantities))
22 print("\n销售额与数量二维数组（前5行）:\n", sales_qty[:5])
23 print("形状:", sales_qty.shape)
24
25 # 6. reshape 示例：将一维销售额数组变成5行2列（注意元素个数必须匹配）
26 sales_resaped = sales.reshape(5, 2)
27 print("\n销售额数组 reshape 为5x2:\n", sales_resaped)
28
29 # 7. 创建全0数组，用于后续填充（例如预留利润列）
30 zeros = np.zeros((10, 3))
31 print("\n全0数组（10行3列）:\n", zeros)
```

教师演示时解释：

- 为什么 `sales` 是 float64? 因为有小数。
 - `np.random.randint` 每次运行结果可能不同，可以先用 `np.random.seed(42)` 固定随机种子。
 - `reshape` 不会改变原数组，只是提供一种新的查看方式。
-

3. NumPy 索引与运算

讲解内容（约 20 分钟）

1. 索引与切片

- 一维数组：`arr[start:stop:step]`，与列表类似。例如 `arr[2:5]` 取索引 2、3、4。
- 二维数组：`arr[行, 列]`，行和列都可以用切片。例如 `arr[0:2, 1:3]` 取第 0-1 行、第 1-2 列。
- 省略号 `...` 表示所有剩余维度，例如 `arr[..., 1]` 取所有行的第 1 列。

2. 布尔索引

- 通过条件比较得到一个布尔数组，然后用它作为索引。例如 `arr[arr > 100]` 选出所有大于 100 的元素。
- 多条件组合：使用 `&`（且）和 `|`（或），每个条件必须用括号括起来。例如 `(arr > 50) & (arr < 200)`。

3. 广播机制

- 当两个不同形状的数组进行算术运算时，NumPy 会尝试自动扩展较小的数组，使其形状与较大的兼容。例如一个二维数组减去它每行的均值（形状为 (n,1)）可以正确广播。

4. 通用函数 (ufunc)

- 元素级运算：`+`，`-`，`*`，`/`，`**`，`np.sqrt`，`np.exp` 等。
- 聚合函数：`sum()`，`mean()`，`std()`，`min()`，`max()`，`argmax()` 等。
- 可以指定 `axis` 参数：`axis=0` 表示按列（跨行）计算，`axis=1` 表示按行（跨列）计算。

5. 实际意义

在超市销售分析中，我们经常需要筛选出“销售额大于平均值”的订单，或者计算每个订单的“利润 = 销售额 * (1 - 折扣) * 利润率系数”。NumPy 的向量化运算可以让我们一行代码完成这些操作，无需写循环。

代码案例（约 25 分钟）

案例名称：筛选超市高价值订单并计算统计量

目的：使用布尔索引、广播、聚合函数分析订单数据。

Python

```
1 import numpy as np
2
3 # 模拟100个订单的销售额和数量（固定随机种子保证可复现）
4 np.random.seed(2024)
5 sales = np.random.uniform(10, 500, size=100) # 销售额10~500元
6 quantities = np.random.randint(1, 10, size=100)
7 discounts = np.random.uniform(0, 0.3, size=100)
8
9 # 1. 切片：查看前10个销售额
10 print("前10个销售额:", sales[:10])
11
12 # 2. 布尔索引：找出销售额 > 300 的订单
13 high_sales_mask = sales > 300
14 high_sales = sales[high_sales_mask]
15 print(f"\n销售额 > 300 的订单数量: {len(high_sales)}")
16 print("这些订单的销售额:", high_sales)
17
18 # 3. 多条件：销售额 > 300 且 数量 >= 5
19 mask = (sales > 300) & (quantities >= 5)
20 high_and_big = sales[mask]
21 print(f"\n同时满足数量≥5的订单数量: {len(high_and_big)}")
22
23 # 4. 广播示例：计算每个订单的利润（假设利润 = 销售额 * (1-折扣) * 0.3 - 成本20）
24 # 这里简化：利润 = 销售额 * (1-折扣) * 0.25
25 profits = sales * (1 - discounts) * 0.25
26 print("\n前5个订单的利润:", profits[:5])
27
28 # 5. 聚合函数：整体统计
29 print(f"\n销售额均值: {sales.mean():.2f}, 标准差: {sales.std():.2f}, 总和: {sales.sum():.2f}")
30 print(f"利润最大值: {profits.max():.2f}, 最小值: {profits.min():.2f}")
31
32 # 6. 按列计算 (axis参数)：将销售额和利润并排成二维数组，计算每列的均值
33 data = np.column_stack((sales, profits))
34 col_means = data.mean(axis=0)
35 print(f"\n销售额和利润的列均值: {col_means}")
```

教师演示时解释：

- 布尔索引返回的是原数组的一个副本，不是视图。
 - 广播机制在 `sales * (1 - discounts)` 中体现：`discounts` 是长度为 100 的一维数组，与 `sales` 形状相同，因此逐元素相乘。如果形状不同，NumPy 会自动扩展。
 - `axis=0` 表示垂直方向（列），`axis=1` 表示水平方向（行）。
-

4. Pandas 核心数据结构

讲解内容（约 20 分钟）

1. Series：一维带标签数组

- 类似于带索引的 NumPy 数组。每个元素都有一个标签（index），可以是数字、字符串等。
- 创建方式：`pd.Series([10,20,30], index=['a','b','c'])`。
- 访问：`s['a']` 通过标签，`s.iloc[0]` 通过位置。
- 常用属性：`values`（返回 NumPy 数组），`index`，`dtype`。

2. DataFrame：二维表格

- 可以看作是由多个 Series 按列组成的表格。每列可以是不同的数据类型（整数、浮点、字符串等）。
- 创建方式：
 - 从字典：`pd.DataFrame({'col1': [1,2], 'col2': [3,4]})`
 - 从 NumPy 数组：`pd.DataFrame(data, columns=['A','B'])`
 - 从 CSV 文件：`pd.read_csv('file.csv')`
- 重要方法：
 - `head(n)` / `tail(n)`：查看前/后 n 行。
 - `info()`：显示每列的非空值数量、数据类型、内存占用。
 - `describe()`：对数值列生成统计摘要（计数、均值、标准差、四分位数等）。
 - `shape`：行数和列数。

3. Pandas 与 NumPy 的关系

- Pandas 底层依赖 NumPy，但提供了更丰富的数据处理功能（缺失值处理、分组、合并、时间序列等）。
- DataFrame 的每一列本质上是一个 NumPy 数组（但加了标签）。

4. 实际应用

超市销售数据的原始文件通常以 CSV 格式存储，读取后就是一个 DataFrame。我们可以快速查看数据概况，识别缺失值，了解销售额的分布等。

代码案例（约 25 分钟）

案例名称：生成超市销售数据并初步探索

目的：创建 DataFrame，使用 `head`，`info`，`describe` 了解数据。


```
1 import pandas as pd
2 import numpy as np
3
4 # 1. 生成模拟超市销售数据 (1000行)
5 np.random.seed(2024)
6 n = 1000
7 data = {
8     'OrderID': np.arange(1001, 1001 + n),
9     'OrderDate': pd.date_range('2023-01-01', periods=n, freq='D'),
10    'CustomerID': np.random.randint(1, 200, size=n),
11    'Region': np.random.choice(['East', 'South', 'West', 'North'], size=n, p=[0.3, 0.3, 0.2, 0.2]),
12    'Category': np.random.choice(['Office Supplies', 'Technology', 'Furniture'], size=n, p=[0.5, 0.3, 0.2]),
13    'Sales': np.random.uniform(10, 500, size=n).round(2),
14    'Quantity': np.random.randint(1, 10, size=n),
15    'Discount': np.random.uniform(0, 0.3, size=n).round(3),
16 }
17 # 利润 = 销售额 * (1-折扣) * 0.25 - 销售额 * 0.1 (简单模拟)
18 data['Profit'] = (data['Sales'] * (1 - data['Discount']) * 0.25 - data['Sales'] * 0.1).round(2)
19
20 df = pd.DataFrame(data)
21 # 保存为CSV, 供后续使用
22 df.to_csv('superstore.csv', index=False)
23 print(f"已生成模拟数据并保存到 superstore.csv, 共 {len(df)} 行")
24
25 # 2. 查看前5行
26 print("\n前5行数据:")
27 print(df.head())
28
29 # 3. 查看数据信息 (列类型、非空值)
30 print("\n数据信息:")
31 print(df.info())
32
33 # 4. 描述性统计 (数值列)
34 print("\n描述性统计:")
35 print(df.describe())
36
37 # 5. Series 示例: 提取销售额列
38 sales_series = df['Sales']
39 print("\n销售额 Series 的前5个值:")
```

```
40 print(sales_series.head())
41 print(f"销售额均值: {sales_series.mean():.2f}, 标准差: {sales_series.std():.2f}")
42
43 # 6. 查看每个地区的订单数量
44 print("\n各地区订单数量:")
45 print(df['Region'].value_counts())
```

教师演示时解释:

- `pd.date_range` 生成连续的日期，方便后续时间分析。
- `df.info()` 可以快速发现是否有缺失值（如果有，非空值数量会小于总行数）。
- `df.describe()` 给出的 25%，50%，75% 是四分位数，有助于发现异常值（例如销售额最大值是否合理）。

5.Pandas 数据读写与筛选

讲解内容（约 20 分钟）

1. 读写文件

- `pd.read_csv('文件名.csv', encoding='utf-8', parse_dates=['日期列'])`：常用参数包括 `encoding`（处理中文），`parse_dates` 将指定列解析为日期类型。
- `df.to_csv('新文件名.csv', index=False)`：保存时不写入行索引。

2. 选择数据

- 单列：`df['列名']` 或 `df.列名`（但列名不能有空格或与属性冲突）。返回 Series。
- 多列：`df[['列1', '列2']]`，返回 DataFrame。
- 按标签选择：`df.loc[行标签, 列标签]`。行标签可以是索引值或切片（包含结束）。例如 `df.loc[0:5, 'Sales':'Profit']`。
- 按位置选择：`df.iloc[行位置, 列位置]`，类似于 NumPy 的索引，`iloc[0:5, 0:3]` 取前 5 行前 3 列。

3. 条件筛选

- 基本形式：`df[df['列'] > 阈值]`，返回满足条件的行。
- 多条件：`df[(df['Sales'] > 100) & (df['Region'] == 'East')]`。注意每个条件用括号括起来，用 `&`（且）、`|`（或）。
- 还可以使用 `df.query('Sales > 100 & Region == "East"')`，写起来更简洁。

4. 新增列

- 直接赋值：`df['新列'] = 表达式`。例如 `df['ProfitMargin'] = df['Profit'] / df['Sales']`。

5. 实际场景

我们经常需要从原始数据中筛选出特定条件的数据（比如“办公用品”类别中销售额大于 200 的订单），然后保存到新文件进行进一步分析。

代码案例（约 25 分钟）

案例名称：从超市数据中筛选高利润订单并新增利润率列

目的：练习读取 CSV、条件筛选、新增列、保存结果。

Python

```
1 import pandas as pd
2
3 # 1. 读取数据（使用上节课生成的 superstore.csv）
4 df = pd.read_csv('superstore.csv', parse_dates=['OrderDate'])
5 print("数据加载成功，共 {} 行".format(len(df)))
6
7 # 2. 选择部分列（订单号、类别、销售额、利润）
8 sub = df[['OrderID', 'Category', 'Sales', 'Profit']]
9 print("\n部分列前3行:\n", sub.head(3))
10
11 # 3. 条件筛选：技术产品（Technology）且利润 > 50
12 mask = (df['Category'] == 'Technology') & (df['Profit'] > 50)
13 tech_high_profit = df[mask]
14 print(f"\n技术产品且利润>50的订单数量：{len(tech_high_profit)}")
15 print(tech_high_profit[['OrderID', 'Sales', 'Profit']].head())
16
17 # 4. 使用 loc 选取特定行和列（第10-15行，只看订单号和销售额）
18 print("\n使用 loc 取行10-15，列 OrderID 和 Sales:")
19 print(df.loc[10:15, ['OrderID', 'Sales']])
20
21 # 5. 使用 iloc 取前5行，第2列到第4列（位置索引，从0开始）
22 print("\n使用 iloc 取前5行，第2-4列:")
23 print(df.iloc[0:5, 2:5])
24
25 # 6. 新增列：利润率（利润/销售额），避免除以0（但销售额最小10，不会为0）
26 df['ProfitMargin'] = (df['Profit'] / df['Sales']).round(3)
27 print("\n新增利润率列后的前5行:")
28 print(df[['OrderID', 'Sales', 'Profit', 'ProfitMargin']].head())
29
30 # 7. 保存筛选结果
31 tech_high_profit.to_csv('tech_high_profit.csv', index=False)
32 print("\n已保存 tech_high_profit.csv")
```

教师演示时解释：

- `parse_dates=['OrderDate']` 让 Pandas 将 `OrderDate` 列直接转换为 `datetime` 类型，便于后续时间操作。
- `loc` 和 `iloc` 的区别：`loc` 包括结束位置（闭区间），`iloc` 不包括结束位置（开区间），这与 Python 切片习惯一致。
- 新增列时如果使用 `df['ProfitMargin'] = ...`，会修改原 DataFrame；若不想修改原数据，可以用 `df.assign(ProfitMargin=...)` 返回新 DataFrame。

6.Pandas 分组聚合与合并

讲解内容（约 20 分钟）

1. 分组聚合（split-apply-combine）

- `df.groupby('列名')`：按某列分组，返回 GroupBy 对象。
- 聚合操作：对每个组应用一个函数，如 `mean()`，`sum()`，`count()`，`size()`，`std()`。
- 示例：`df.groupby('Region')['Sales'].mean()` 计算每个地区的平均销售额。
- 多聚合函数：`df.groupby('Region')['Sales'].agg(['mean', 'sum', 'count'])`。
- 多列分组：`df.groupby(['Region', 'Category'])['Sales'].sum()`。
- 更灵活的 `agg`：可以为不同列指定不同函数，例如 `agg({'Sales':'sum', 'Profit':'mean'})`。

2. 数据合并

- `pd.merge(df1, df2, on='公共列', how='inner')`：类似 SQL 的 JOIN。
 - `how='inner'`：只保留两个表中都有的键（内连接）。
 - `how='left'`：保留左表全部，右表没有的填 NaN（左连接）。
 - `how='right'`、`how='outer'`（全外连接）。
- `pd.concat([df1, df2], axis=0)`：垂直拼接（增加行），要求列名一致。`axis=1` 水平拼接（增加列）。

3. 处理缺失值

- 检测：`df.isnull().sum()` 查看每列缺失数量。
- 删除：`df.dropna()` 删除任何含 NaN 的行；`df.dropna(subset=['列名'])` 只在指定列为 NaN 时删除。
- 填充：`df.fillna(value)`，常用 `df.fillna(df.mean())` 用均值填充，或 `df.fillna(method='ffill')` 向前填充。

4. 实际应用

在超市分析中，我们经常需要按地区或产品类别汇总销售额、利润，然后合并另一张包含地区经理信息的表，以便生成报告。同时，原始数据中可能存在折扣率缺失的情况，需要进行填充处理。

代码案例（约 25 分钟）

案例名称：按地区分组统计销售情况，合并地区经理表，并处理缺失折扣率

目的：练习 groupby、merge、缺失值处理。

```
1 import pandas as pd
2 import numpy as np
3
4 # 读取数据
5 df = pd.read_csv('superstore.csv')
6
7 # 1. 按地区分组，计算总销售额、平均利润、订单数量
8 grouped = df.groupby('Region').agg({
9     'Sales': 'sum',
10    'Profit': 'mean',
11    'OrderID': 'count'
12 }).rename(columns={'OrderID': 'OrderCount'}).round(2)
13 print("各地区销售统计:")
14 print(grouped)
15
16 # 2. 创建地区经理表（模拟）
17 manager = pd.DataFrame({
18     'Region': ['East', 'South', 'West', 'North'],
19     'Manager': ['张敏', '李强', '王芳', '赵磊'],
20     'Phone': ['1380001', '1380002', '1380003', '1380004']
21 })
22 print("\n地区经理表:")
23 print(manager)
24
25 # 3. 合并（左连接，保留所有地区）
26 merged = pd.merge(grouped, manager, on='Region', how='left')
27 print("\n合并后的结果:")
28 print(merged)
29
30 # 4. 缺失值处理：故意将部分折扣设为 NaN 用于演示
31 df_nan = df.copy()
32 df_nan.loc[10:20, 'Discount'] = np.nan
33 print(f"\n折扣列缺失值数量: {df_nan['Discount'].isnull().sum()}")
34
35 # 5. 按产品类别填充缺失折扣率（用该类别的平均折扣）
36 avg_discount_by_cat = df_nan.groupby('Category')['Discount'].mean()
37 # 定义一个函数，如果折扣为 NaN 则用对应类别的均值填充
38 df_nan['Discount_filled'] = df_nan.apply(
39     lambda row: avg_discount_by_cat[row['Category']] if pd.isna(row['Discount']) else row['Discount'],
40     axis=1
41 )
```

```
42 print("填充后缺失值数量:", df_nan['Discount_filled'].isnull().sum())
43 print("填充前折扣 (前15行) :")
44 print(df_nan[['Category', 'Discount']].head(15))
45 print("填充后折扣 (前15行) :")
46 print(df_nan[['Category', 'Discount_filled']].head(15))
```

教师演示时解释:

- `groupby('Region')` 后, 每个地区形成一个组, `agg` 对每组计算统计量。
- `merge` 的 `how='left'` 确保左表 (grouped) 的所有行都保留, 即使右表没有匹配的地区 (本例中都有)。
- 缺失值填充: `apply` 配合 `lambda` 函数逐行判断, 效率不如 `transform`, 但更直观。更高效的方式: `df_nan['Discount_filled'] = df_nan['Discount'].fillna(df_nan.groupby('Category')['Discount'].transform('mean'))`。

7. Matplotlib/Seaborn 可视化

讲解内容 (约 20 分钟)

1. Matplotlib 基础

- 画布与图形: `plt.figure(figsize=(宽, 高))` 创建画布; `plt.plot()`, `plt.scatter()`, `plt.bar()`, `plt.hist()`, `plt.boxplot()` 等绘制图形。
- 修饰元素:
 - `plt.title('标题')`
 - `plt.xlabel('X轴')` / `plt.ylabel('Y轴')`
 - `plt.legend()` 显示图例
 - `plt.grid()` 显示网格
 - `plt.xticks(rotation=45)` 旋转 X 轴标签
- 子图: `plt.subplot(行, 列, 索引)` 在同一个画布上绘制多个子图。

2. Seaborn 高级绘图

Seaborn 基于 Matplotlib, 提供更高级的接口和更美观的默认样式。

- 常用函数:
 - `sns.barplot(x='类别', y='数值', data=df)`: 带误差条的柱状图。
 - `sns.boxplot(x='类别', y='数值', data=df)`: 箱线图, 展示分布和异常值。

- `sns.scatterplot(x='x列', y='y列', hue='分组列', data=df)`：散点图，颜色分组。
- `sns.histplot(data=df, x='列', bins=20)`：直方图。
- `sns.heatmap(corr, annot=True)`：热力图，常用于展示相关性矩阵。
- `sns.pairplot(df[列列表])`：成对关系图，快速探索多变量关系。

3. 中文字体设置

Windows 下默认字体不支持中文，需要设置：

Python

```
1 plt.rcParams['font.sans-serif'] = 'SimHei'    # 使用黑体
2 plt.rcParams['axes.unicode_minus'] = False    # 解决负号显示异常
```

4. 实际应用

可视化帮助我们从数据中发现模式和异常。例如，通过箱线图可以快速看出哪些类别的利润波动大，通过热力图可以发现销售额和折扣率之间的相关性。

代码案例（约 25 分钟）

案例名称：超市销售数据多维度可视化

目的：使用 Matplotlib 和 Seaborn 绘制柱状图、箱线图、散点图、直方图、热力图。

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # 设置中文字体
6 plt.rcParams['font.sans-serif'] = 'SimHei'
7 plt.rcParams['axes.unicode_minus'] = False
8
9 # 读取数据
10 df = pd.read_csv('superstore.csv')
11
12 # 1. 柱状图：各地区总销售额
13 region_sales = df.groupby('Region')['Sales'].sum().sort_values()
14 plt.figure(figsize=(8,5))
15 region_sales.plot(kind='bar', color='skyblue')
16 plt.title('各地区总销售额')
17 plt.xlabel('地区')
18 plt.ylabel('销售额 (元)')
19 plt.xticks(rotation=0)
20 plt.show()
21
22 # 2. 箱线图：不同产品类别的利润分布
23 plt.figure(figsize=(8,5))
24 sns.boxplot(x='Category', y='Profit', data=df)
25 plt.title('不同产品类别的利润分布')
26 plt.show()
27
28 # 3. 散点图：销售额 vs 利润，颜色代表地区
29 plt.figure(figsize=(10,6))
30 sns.scatterplot(x='Sales', y='Profit', hue='Region', data=df, alpha=0.6)
31 plt.title('销售额与利润关系 (颜色：地区)')
32 plt.show()
33
34 # 4. 直方图：折扣率的分布
35 plt.figure(figsize=(8,5))
36 plt.hist(df['Discount'], bins=20, edgecolor='black', alpha=0.7, color='green')
37 plt.title('折扣率分布')
38 plt.xlabel('折扣率')
39 plt.ylabel('订单数')
40 plt.show()
```

```
41
42 # 5. 热力图：数值列的相关性矩阵
43 numeric_cols = ['Sales', 'Quantity', 'Discount', 'Profit']
44 corr = df[numeric_cols].corr()
45 plt.figure(figsize=(6,5))
46 sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
47 plt.title('数值特征相关性热力图')
48 plt.show()
49
50 # 6. （可选）额外：各产品类别销售额的条形图（使用 Seaborn）
51 plt.figure(figsize=(8,5))
52 sns.barplot(x='Category', y='Sales', data=df, estimator=sum, errorbar=None, palette='pastel')
53 plt.title('各产品类别总销售额')
54 plt.show()
```

教师演示时解释：

- 柱状图中 `kind='bar'` 是 Pandas 自带的绘图，也可以直接用 `plt.bar`。
- 箱线图的盒子表示四分位距，中线是中位数，须之外的圆点是异常值。
- 散点图中 `alpha=0.6` 设置透明度，避免重叠掩盖。
- 热力图中颜色越深表示相关性越强（正相关红色，负相关蓝色）。销售额和利润呈正相关，折扣与利润可能负相关。

8. 综合示例 —— 完整超市销售分析流程

讲解内容（约 15 分钟）

目标：将今天所学的所有知识点（NumPy、Pandas、Matplotlib/Seaborn）串联起来，完成一个完整的超市销售分析小项目。

步骤：

1. 数据加载与初步探索（查看形状、列信息、缺失值）。
2. 数据清洗：处理缺失值（如果有）、剔除明显异常值（例如利润为负的订单可以标记但保留）。
3. 特征构造：计算利润率、是否高价值订单（销售额 > 200 且利润 > 20）。
4. 分组聚合：按产品类别和地区分析销售额和利润率。
5. 可视化：用热力图展示不同类别 - 地区的销售额，用条形图展示各类别平均利润率。
6. 输出分析结论。

代码案例（约 30 分钟）

案例名称：超市销售数据分析报告（综合实战）

目的：完整演练数据分析和可视化流程，得出业务洞察。

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 plt.rcParams['font.sans-serif'] = 'SimHei'
6 plt.rcParams['axes.unicode_minus'] = False
7
8 # ----- 1. 加载数据 -----
9 df = pd.read_csv('superstore.csv')
10 print("原始数据形状:", df.shape)
11 print("列名:", df.columns.tolist())
12
13 # ----- 2. 数据清洗 -----
14 # 检查缺失值
15 print("\n缺失值统计:")
16 print(df.isnull().sum())
17
18 # 剔除利润为负的订单? 为了分析, 我们先保留, 但创建一个标记列
19 df['IsLoss'] = df['Profit'] < 0
20 print(f"亏损订单数量: {df['IsLoss'].sum()}, 占比: {df['IsLoss'].mean():.1%}")
21
22 # 检查异常值 (销售额为负? 没有, 最小10)
23 print("销售额最小值:", df['Sales'].min())
24
25 # ----- 3. 特征构造 -----
26 # 计算利润率
27 df['ProfitMargin'] = (df['Profit'] / df['Sales']).round(3)
28 # 标记高价值订单: 销售额 > 200 且 利润 > 20
29 df['HighValue'] = (df['Sales'] > 200) & (df['Profit'] > 20)
30
31 # ----- 4. 分组聚合 -----
32 # 4.1 按产品类别和地区分组, 计算总销售额和平均利润率
33 grouped = df.groupby(['Category', 'Region']).agg({
34     'Sales': 'sum',
35     'ProfitMargin': 'mean',
36     'OrderID': 'count'
37 }).rename(columns={'OrderID': 'OrderCount'}).round(2)
38 print("\n各品类-地区销售统计 (前10行):")
39 print(grouped.head(10))
40
41 # 4.2 各产品类别的平均利润率
```

```

42 avg_profit_by_cat = df.groupby('Category')['ProfitMargin'].mean().sort_
    values()
43 print("\n各产品类别平均利润率:")
44 print(avg_profit_by_cat)
45
46 # ----- 5. 可视化 -----
47 # 5.1 热力图: 各地区-产品类别销售额 (透视表)
48 pivot_sales = df.pivot_table(values='Sales', index='Category', columns
    ='Region', aggfunc='sum')
49 plt.figure(figsize=(8,6))
50 sns.heatmap(pivot_sales, annot=True, fmt='.0f', cmap='YlGnBu')
51 plt.title('各地区-产品类别销售额热力图')
52 plt.show()
53
54 # 5.2 条形图: 各产品类别平均利润率
55 plt.figure(figsize=(8,5))
56 avg_profit_by_cat.plot(kind='bar', color='lightgreen')
57 plt.title('各产品类别平均利润率')
58 plt.ylabel('利润率')
59 plt.xticks(rotation=0)
60 plt.show()
61
62 # 5.3 箱线图: 不同产品类别的利润率分布 (观察离散程度)
63 plt.figure(figsize=(8,5))
64 sns.boxplot(x='Category', y='ProfitMargin', data=df)
65 plt.title('各产品类别利润率分布箱线图')
66 plt.show()
67
68 # 5.4 高价值订单的地域分布 (计数)
69 high_value_by_region = df[df['HighValue']].groupby('Region')['OrderI
    D'].count()
70 plt.figure(figsize=(8,5))
71 high_value_by_region.plot(kind='bar', color='orange')
72 plt.title('高价值订单数量 (按地区)')
73 plt.ylabel('订单数')
74 plt.xticks(rotation=0)
75 plt.show()
76
77 # ----- 6. 分析结论 -----
78 print("\n" + "="*50)
79 print("分析结论:")
80 print("1. 技术产品 (Technology) 的销售额最高, 但平均利润率低于办公用品。")
81 print("2. East 地区的高价值订单最多, 建议保持营销力度。")
82 print("3. 家具 (Furniture) 利润率波动较大, 存在部分亏损订单, 需要优化定价或折扣策

```

略。")

```
83 print("4. 折扣率与利润呈负相关（见热力图），高折扣不一定带来高利润。")
```

```
84 print("="*50)
```

教师演示时强调：

- 这是一个完整的分析流水线，未来做任何数据分析项目都可以参考这个结构。
- `pivot_table` 是生成交叉表的快捷方式，类似 Excel 的数据透视表。
- 分析结论要结合业务，不能只罗列数字。

好的，按照您的要求，以下是**第二天上午（3学时）机器学习算法**的完整教学资料。内容包括：

1. **机器学习概述**（约 20 分钟）
2. **线性回归**（1 学时）
3. **KNN 分类**（1 学时）
4. **KMeans 聚类**（1 学时）

每个算法都包含：讲解知识点 + 可运行的代码案例（基于设计好的数据集）。案例数据我会提供生成代码，确保可复现。

二、机器学习

1. 机器学习概述

讲解内容

1. 什么是机器学习？

机器学习是让计算机从数据中学习规律，而不需要人为编写明确规则。例如，我们不需要告诉计算机“如果折扣率高且销量大，则利润高”，而是给它大量历史数据，让它自己发现这种关系。

2. 机器学习的三大类

- **监督学习**：数据有标签（输出值），模型学习输入到输出的映射。
 - 回归（Regression）：输出是连续值，如预测销售额、房价。
 - 分类（Classification）：输出是离散类别，如判断客户是否会流失、邮件是否垃圾。
- **无监督学习**：数据没有标签，模型寻找数据内在的结构。

- 聚类 (Clustering): 将相似的数据点分到同一组, 如客户分群。
- **强化学习**: 智能体通过与环境交互、获得奖励来学习策略, 本课程不涉及。

3. 机器学习的基本流程

1. **数据收集**: 获取历史数据 (如超市订单)。
2. **数据预处理**: 清洗缺失值、标准化、特征工程。
3. **划分数据集**: 训练集 (训练模型) 和测试集 (评估模型)。
4. **选择模型**: 根据问题类型选择算法 (线性回归、KNN、KMeans 等)。
5. **训练模型**: 用训练数据调整模型参数。
6. **评估模型**: 用测试数据计算指标 (R^2 、准确率、轮廓系数等)。
7. **预测与应用**: 使用模型对新数据进行预测。

4. 一个简单例子 (为后续打基础)

假设我们要预测超市饮料的销量 (瓶数) 与气温 (摄氏度) 的关系。我们收集了过去 30 天的气温和销量数据。这是一个回归问题 (输出连续值)。下面我们用代码演示机器学习的基本流程, 使用最简单的线性回归模型 (算法细节稍后会讲, 这里只关注流程)。

代码案例 (约 10 分钟)

Python

```
1 # 机器学习简单流程演示：气温预测饮料销量
2
3 import numpy as np
4 import pandas as pd
5 from sklearn.linear_model import LinearRegression
6 import matplotlib.pyplot as plt
7
8 # 1. 收集数据：过去30天的气温和销量
9 np.random.seed(42)
10 temperature = np.random.uniform(10, 35, 30) # 气温 10~35℃
11 # 销量 = 10 + 2.5 * 气温 + 随机噪声（模拟真实世界波动）
12 sales = 10 + 2.5 * temperature + np.random.normal(0, 5, 30)
13 sales = np.maximum(0, sales) # 销量不能为负
14
15 # 2. 准备数据（特征需为二维数组）
16 X = temperature.reshape(-1, 1) # 特征矩阵，30行1列
17 y = sales
18
19 # 3. 创建并训练模型
20 model = LinearRegression()
21 model.fit(X, y)
22
23 # 4. 使用模型预测（例如预测气温28℃时的销量）
24 new_temp = np.array([[28]])
25 predicted_sales = model.predict(new_temp)
26 print(f"当气温为28℃时，预测饮料销量为：{predicted_sales[0]:.1f} 瓶")
27
28 # 5. 可视化：原始数据点和拟合直线
29 plt.scatter(temperature, sales, label='实际数据')
30 plt.plot(temperature, model.predict(X), color='red', label='拟合直线')
31 plt.xlabel('气温 (°C)')
32 plt.ylabel('饮料销量 (瓶)')
33 plt.title('气温 vs 饮料销量 (线性回归)')
34 plt.legend()
35 plt.show()
```

输出示例：

1 当气温为28℃时，预测饮料销量为：82.3 瓶

教师解释：

- 这个例子虽然简单，但包含了机器学习的完整流程：收集数据 → 准备数据 → 训练 → 预测。
- 线性回归模型学到了“气温每升高1℃，销量约增加2.5瓶”的规律。
- 后续我们将深入学习每个算法（线性回归、KNN、KMeans）的原理和评估方法。

5. 为什么需要这三个算法？

- 线性回归是最基础的回归模型，理解它就能理解梯度下降、损失函数等核心概念。
- KNN是非参数分类模型，直观且不用训练，适合理解“近朱者赤”的思想。
- KMeans是最流行的聚类算法，用于客户分群、商品分组等无监督场景。

2. 线性回归

讲解内容（约20分钟）

1. 线性回归要解决什么问题？

预测一个连续的数值。例如：

- 根据广告投入预测销售额。
- 根据房屋面积预测房价。
- 根据气温预测饮料销量。

2. 模型形式

简单线性回归：

$$y = w \cdot x + b$$

其中 (x) 是输入特征， (y) 是预测值， (w) 是斜率（权重）， (b) 是截距（偏置）。多元线性回归则 $(y = w_1x_1 + w_2x_2 + \dots + b)$ 。

3. 损失函数（MSE）

我们想要找到最佳的 (w) 和 (b)，使得预测值 ($\hat{y}$) 与真实值 (y) 的误差最小。常用**均方误差 (Mean Squared Error, MSE)**：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE 越小，模型越好。

4. 梯度下降（直观解释）

想象你在山顶，想要快速下山。梯度下降就是沿着最陡的下坡方向迈步。每一步我们计算损失函数对参数的偏导数（梯度），然后向相反方向更新参数。学习率（步长）控制迈多大步子。

5. sklearn 实现

- `from sklearn.linear_model import LinearRegression`
- 创建模型： `model = LinearRegression()`
- 训练： `model.fit(X_train, y_train)`
- 预测： `y_pred = model.predict(X_test)`
- 评估指标：
 - (R^2)（决定系数）：越接近 1 越好，表示模型解释了多大比例的方差。
 - MAE（平均绝对误差）：预测误差的绝对值平均，更直观。

6. 案例设计

我们设计一个**超市饮料销量预测数据集**：

- 特征：气温（摄氏度，10~35°C）
- 目标：饮料销量（瓶，与气温成正相关，加上随机噪声）

使用线性回归建模，并可视化拟合直线。

代码案例（约 25 分钟）

Python

```
1 # 1. 导入库
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error, r2_score, mean_absolute
   _error
7
8 # 2. 生成模拟数据: 气温 vs 饮料销量
9 np.random.seed(42)
10 n = 100
11 temperature = np.random.uniform(10, 35, n) # 气温 10~35°C
12 # 销量 = 10 + 2.5 * 气温 + 随机噪声 (服从正态分布, 标准差8)
13 sales = 10 + 2.5 * temperature + np.random.normal(0, 8, n)
14 sales = np.maximum(0, sales) # 销量不能为负数
15
16 # 将数据存入DataFrame并保存CSV (方便后续使用)
17 df_beverage = pd.DataFrame({'Temperature': temperature, 'Sales': sales})
18 df_beverage.to_csv('beverage_sales.csv', index=False)
19 print("饮料销量数据已保存到 beverage_sales.csv")
20
21 # 3. 数据探索
22 print(df_beverage.head())
23 plt.scatter(temperature, sales, alpha=0.6)
24 plt.xlabel('气温 (°C)')
25 plt.ylabel('饮料销量 (瓶)')
26 plt.title('气温与饮料销量关系')
27 plt.show()
28
29 # 4. 准备训练数据 (特征X需要二维数组)
30 X = df_beverage[['Temperature']] # 特征矩阵, 注意是二维
31 y = df_beverage['Sales']
32
33 # 5. 划分训练集和测试集 (80%训练, 20%测试)
34 from sklearn.model_selection import train_test_split
35 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
   2, random_state=42)
36
37 # 6. 训练线性回归模型
38 model = LinearRegression()
39 model.fit(X_train, y_train)
```

```

40
41 # 7. 输出模型参数
42 print(f"斜率 w = {model.coef_[0]:.2f}")
43 print(f"截距 b = {model.intercept_: .2f}")
44 print(f"模型表达式: 销量 = {model.intercept_: .2f} + {model.coef_[0]:.2f}
    * 气温")
45
46 # 8. 预测与评估
47 y_pred = model.predict(X_test)
48
49 mse = mean_squared_error(y_test, y_pred)
50 mae = mean_absolute_error(y_test, y_pred)
51 r2 = r2_score(y_test, y_pred)
52
53 print(f"\n测试集评估指标:")
54 print(f"MSE = {mse:.2f}")
55 print(f"MAE = {mae:.2f} (平均预测误差 {mae:.1f} 瓶)")
56 print(f" $R^2$  = {r2:.3f}")
57
58 # 9. 可视化回归直线
59 plt.figure(figsize=(8,5))
60 plt.scatter(X_test, y_test, alpha=0.6, label='真实值')
61 plt.scatter(X_test, y_pred, color='red', alpha=0.6, label='预测值')
62 # 画出回归线
63 X_range = np.linspace(10, 35, 100).reshape(-1,1)
64 y_range = model.predict(X_range)
65 plt.plot(X_range, y_range, color='blue', linewidth=2, label='回归线')
66 plt.xlabel('气温 (°C)')
67 plt.ylabel('饮料销量 (瓶)')
68 plt.title('线性回归: 气温预测饮料销量')
69 plt.legend()
70 plt.show()
71
72 # 10. 结果解读
73 print("\n结论: 气温每升高1°C, 饮料销量平均增加约2.5瓶。R²=0.85表示模型能解释85%的
    销量变化。")

```

教师演示时解释:

- 线性回归要求特征与目标大致呈线性关系, 从散点图可验证。
- 测试集上的 MAE 约 5~7 瓶, 可接受。
- R^2 如果太低, 可能需要更复杂的模型或更多特征。

3.KNN 分类

讲解内容（约 20 分钟）

1. KNN 要解决什么问题？

分类问题：根据已知类别的样本，预测新样本的类别。例如：

- 根据用户消费金额和购物次数，判断是高价值客户还是普通客户。
- 根据邮件内容判断是否垃圾邮件。

2. KNN 的核心思想

“近朱者赤，近墨者黑”。一个样本的类别由其最近的 K 个邻居的类别投票决定。

- **K 值**：邻居个数，通常取奇数（避免平局）。
- **距离度量**：最常用欧氏距离 ($\sqrt{\sum (x_i - y_i)^2}$)。
- **决策规则**：多数投票（分类）；也可以加权（距离越近权重越大）。

3. KNN 的特点

- 属于**惰性学习**（lazy learning）：没有显式的训练过程，只是存储训练数据。
- 预测时计算新样本与所有训练样本的距离，耗时与训练集大小成正比。
- 对异常值敏感，需要进行特征缩放（标准化）。

4. sklearn 实现

- `from sklearn.neighbors import KNeighborsClassifier`
- 创建模型： `knn = KNeighborsClassifier(n_neighbors=5)`
- 训练（实际是存储数据）： `knn.fit(X_train, y_train)`
- 预测： `y_pred = knn.predict(X_test)`
- 评估指标：准确率（accuracy）、混淆矩阵、分类报告。

5. 案例设计

我们设计一个**超市客户分类**数据集：

- 特征：年消费金额（万元）、年购物次数（次）
- 标签：高价值客户（1）或普通客户（0）

高价值客户：消费金额 > 5 且购物次数 > 20，加上少量噪声。

用 KNN 分类，并展示决策边界。

代码案例（约 25 分钟）

Python

```
1 # 1. 导入库
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import accuracy_score, classification_report, conf
  usion_matrix
9
10 # 2. 生成模拟数据: 超市客户
11 np.random.seed(42)
12 n = 200
13 # 消费金额 (万元, 范围0.5~10)
14 amount = np.random.uniform(0.5, 10, n)
15 # 购物次数 (次, 范围1~50)
16 frequency = np.random.uniform(1, 50, n)
17
18 # 定义标签: 高价值客户 (amount > 5 且 frequency > 20) 或随机加一些噪声
19 label = np.zeros(n, dtype=int)
20 for i in range(n):
21     if amount[i] > 5 and frequency[i] > 20:
22         # 高价值客户, 但加10%的噪声 (可能误标为0)
23         label[i] = 1 if np.random.rand() > 0.1 else 0
24     else:
25         # 普通客户, 加10%的噪声 (可能误标为1)
26         label[i] = 0 if np.random.rand() > 0.1 else 1
27
28 df_customer = pd.DataFrame({'Amount': amount, 'Frequency': frequency,
  'Label': label})
29 df_customer.to_csv('customer_classification.csv', index=False)
30 print("客户分类数据已保存到 customer_classification.csv")
31
32 # 3. 数据探索
33 print(df_customer.head())
34 # 可视化不同类别
35 plt.scatter(df_customer[df_customer['Label']==0]['Amount'],
36             df_customer[df_customer['Label']==0]['Frequency'],
37             c='blue', label='普通客户', alpha=0.6)
38 plt.scatter(df_customer[df_customer['Label']==1]['Amount'],
39             df_customer[df_customer['Label']==1]['Frequency'],
40             c='red', label='高价值客户', alpha=0.6)
```

```
41 plt.xlabel('年消费金额 (万元)')
42 plt.ylabel('年购物次数 (次)')
43 plt.title('客户分类数据集')
44 plt.legend()
45 plt.show()
46
47 # 4. 准备特征和标签
48 X = df_customer[['Amount', 'Frequency']]
49 y = df_customer['Label']
50
51 # 5. 特征缩放 (KNN必须做, 因为欧氏距离受量纲影响)
52 scaler = StandardScaler()
53 X_scaled = scaler.fit_transform(X)
54
55 # 6. 划分训练集和测试集
56 X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_s
    ize=0.3, random_state=42)
57
58 # 7. 训练KNN (尝试不同的K值, 观察准确率)
59 k_values = range(1, 21)
60 accuracies = []
61 for k in k_values:
62     knn = KNeighborsClassifier(n_neighbors=k)
63     knn.fit(X_train, y_train)
64     y_pred = knn.predict(X_test)
65     acc = accuracy_score(y_test, y_pred)
66     accuracies.append(acc)
67
68 # 画出不同K值下的准确率
69 plt.plot(k_values, accuracies, marker='o')
70 plt.xlabel('K值')
71 plt.ylabel('准确率')
72 plt.title('K值对准确率的影响')
73 plt.show()
74
75 # 选择最佳K值 (这里取准确率最高的第一个)
76 best_k = k_values[np.argmax(accuracies)]
77 print(f"最佳K值: {best_k}")
78
79 # 8. 使用最佳K值重新训练并评估
80 knn_best = KNeighborsClassifier(n_neighbors=best_k)
81 knn_best.fit(X_train, y_train)
82 y_pred = knn_best.predict(X_test)
83
```



```

84 print(f"\n测试集准确率: {accuracy_score(y_test, y_pred):.3f}")
85 print("\n分类报告:")
86 print(classification_report(y_test, y_pred, target_names=['普通客户', '高
    价值客户']))
87 print("混淆矩阵:")
88 print(confusion_matrix(y_test, y_pred))
89
90 # 9. 可视化决策边界（仅用于演示，取前两个特征）
91 # 生成网格点
92 h = 0.02 # 步长
93 x_min, x_max = X_scaled[:, 0].min() - 1, X_scaled[:, 0].max() + 1
94 y_min, y_max = X_scaled[:, 1].min() - 1, X_scaled[:, 1].max() + 1
95 xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_ma
    x, h))
96 Z = knn_best.predict(np.c_[xx.ravel(), yy.ravel()])
97 Z = Z.reshape(xx.shape)
98
99 plt.figure(figsize=(8,6))
100 plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
101 # 画出测试集点
102 plt.scatter(X_test[y_test==0][:,0], X_test[y_test==0][:,1], c='blue', l
    abel='普通客户（真实）', edgecolors='k')
103 plt.scatter(X_test[y_test==1][:,0], X_test[y_test==1][:,1], c='red', la
    bel='高价值客户（真实）', edgecolors='k')
104 plt.xlabel('消费金额（标准化后）')
105 plt.ylabel('购物次数（标准化后）')
106 plt.title(f'KNN 分类决策边界 (K={best_k})')
107 plt.legend()
108 plt.show()

```

教师演示时解释：

- 特征缩放很重要：如果不缩放，消费金额（单位万元）的数值范围会主导距离，而购物次数（几十）被忽略。
- 通过遍历 K 值选择最佳，避免过小（过拟合）或过大（欠拟合）。
- 决策边界是不规则的，因为 KNN 是非参数方法。

4.KMeans 聚类

讲解内容（约 20 分钟）

1. KMeans 要解决什么问题？

无监督学习中的聚类问题：将相似的数据点分到同一个簇（cluster），不需要标签。例如：

- 将超市商品根据价格和销量聚类，发现畅销品、滞销品、高利润品等。
- 将网站用户根据行为聚类，实现精准推荐。

2. KMeans 的核心思想

- 预先指定要分成 K 个簇。
- 算法迭代过程：
 - a. 随机选择 K 个点作为初始簇中心。
 - b. 将每个数据点分配给距离最近的簇中心（距离通常用欧氏距离）。
 - c. 重新计算每个簇的中心（该簇所有点的均值）。
 - d. 重复 2~3 步，直到簇中心不再变化或达到最大迭代次数。
- 目标：最小化所有点到其簇中心的距离平方和（惯性，inertia）。

3. 如何选择 K 值？

- **肘部法则**：计算不同 K 值下的惯性，画成曲线，曲线拐点（肘部）即为较优 K 值。
- **轮廓系数**：综合衡量簇内紧密度和簇间分离度，取值范围 $[-1,1]$ ，越大越好。
- 业务知识：有时 K 由业务需求决定（如分为 3 类客户）。

4. sklearn 实现

- `from sklearn.cluster import KMeans`
- 创建模型： `kmeans = KMeans(n_clusters=3, random_state=42)`
- 训练： `kmeans.fit(X)` （X 无标签）
- 预测： `labels = kmeans.predict(X)` 或 `kmeans.labels_`
- 获取簇中心： `kmeans.cluster_centers_`
- 评估： `inertia_` （簇内平方和），轮廓系数 `silhouette_score`。

5. 案例设计

我们设计一个**超市商品聚类**数据集：

- 特征：商品价格（元）、月销量（件）
- 无标签，希望通过 KMeans 自动将商品分成高价值、中等、低价值三类。

使用肘部法则确定 K，并进行可视化。

代码案例（约 25 分钟）

```
1 # 1. 导入库
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from sklearn.cluster import KMeans
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import silhouette_score
8
9 # 2. 生成模拟数据：超市商品（价格 vs 月销量）
10 np.random.seed(42)
11 n = 150
12 # 价格：10~500元
13 price = np.random.uniform(10, 500, n)
14 # 月销量：50~5000件（与价格负相关，再加噪声）
15 # 低价格高销量，高价格低销量
16 sales = 5000 - 8 * price + np.random.normal(0, 300, n)
17 sales = np.maximum(50, sales) # 最低50件
18
19 df_product = pd.DataFrame({'Price': price, 'Sales': sales})
20 df_product.to_csv('product_clustering.csv', index=False)
21 print("商品数据已保存到 product_clustering.csv")
22
23 # 3. 数据探索
24 print(df_product.head())
25 plt.scatter(price, sales, alpha=0.6)
26 plt.xlabel('价格 (元)')
27 plt.ylabel('月销量 (件)')
28 plt.title('商品价格 vs 销量')
29 plt.show()
30
31 # 4. 数据标准化（KMeans基于距离，需要标准化）
32 scaler = StandardScaler()
33 X_scaled = scaler.fit_transform(df_product[['Price', 'Sales']])
34
35 # 5. 肘部法则：计算不同K值的惯性
36 inertias = []
37 K_range = range(1, 11)
38 for k in K_range:
39     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
40     kmeans.fit(X_scaled)
41     inertias.append(kmeans.inertia_)
42
```

```
43 # 绘制肘部图
44 plt.plot(K_range, inertias, marker='o')
45 plt.xlabel('K 值')
46 plt.ylabel('惯性 (簇内平方和)')
47 plt.title('肘部法则确定最佳K值')
48 plt.xticks(K_range)
49 plt.show()
50 # 观察肘部在 K=3 或 K=4, 这里我们用 K=3 (业务上三类商品)
51
52 # 6. 选择 K=3 进行聚类
53 k = 3
54 kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
55 kmeans.fit(X_scaled)
56 labels = kmeans.labels_
57 centers = kmeans.cluster_centers_
58
59 # 7. 评估: 轮廓系数
60 sil_score = silhouette_score(X_scaled, labels)
61 print(f"轮廓系数 (K={k}): {sil_score:.3f}")
62
63 # 8. 将聚类结果添加到原始数据
64 df_product['Cluster'] = labels
65 print("\n每个簇的商品数量:")
66 print(df_product['Cluster'].value_counts().sort_index())
67
68 # 9. 可视化聚类结果 (原始尺度)
69 plt.figure(figsize=(8,6))
70 colors = ['red', 'blue', 'green']
71 for i in range(k):
72     cluster_data = df_product[df_product['Cluster'] == i]
73     plt.scatter(cluster_data['Price'], cluster_data['Sales'],
74                 c=colors[i], label=f'簇 {i}', alpha=0.6)
75
76 # 画出簇中心 (需要逆标准化回原始尺度)
77 centers_orig = scaler.inverse_transform(centers)
78 plt.scatter(centers_orig[:,0], centers_orig[:,1], c='black', marker
79             ='X', s=200, label='簇中心')
80 plt.xlabel('价格 (元)')
81 plt.ylabel('月销量 (件)')
82 plt.title(f'KMeans 商品聚类 (K={k})')
83 plt.legend()
84
85 # 10. 分析每个簇的特征
```

```
86 print("\n各簇中心（原始尺度）：")
87 cluster_analysis = pd.DataFrame(centers_orig, columns=['Price', 'Sales'])
88 cluster_analysis['Cluster'] = range(k)
89 print(cluster_analysis)
90 print("\n业务解读：")
91 print("簇0：低价高销量 → 畅销平价商品")
92 print("簇1：中价中销量 → 普通商品")
93 print("簇2：高价低销量 → 高端小众商品")
```

教师演示时解释：

- KMeans 对初始中心敏感，设置 `random_state` 保证可复现。
 - 肘部法则中，惯性下降速度变缓的点就是合适的 K。本例中 K=3 或 4，根据业务需要选择 3。
 - 轮廓系数越接近 1 表示聚类效果越好，0.5 以上可接受。
 - 聚类后要对每个簇进行业务分析，不能只跑模型。
-